

CS 223: Systems Programming

Instructor: Kelly Shiptoski

Lecture: 1, Week: 1

Agenda

Introductions

Admin

What is a **computer system**?

Development environment overview

Comparing C and Java

Makefiles

Unix review / working with paths

Editors

Please Sign In

Write your name, email address, and student ID number on the sheet.

Thank you!

Prof. Kelly Shiptoski

PhD from UPenn.

Worked at multiple startups post graduation.

Research areas: operating systems, containers / virtual machines, determinism, reproducibility.

Interested in Research?

If you find what you learn in this class interesting, consider trying out *systems research*.

Looking for undergraduate students to work on various research projects with me (and with collaboration from some of my colleagues from UPenn).

If you are interested, we can set up a time to talk about the possibility.

Note: this work would start in the spring semester.

Class Resources

Textbook: Dive into Systems (don't buy it, I will post a link to a free version in our class slack / website).

Slack: Announcements, links, etc.

Website: Policies, syllabus, schedule, office hours info, etc.

Github: code repo.

Lab: Park 231.

Links

Textbook: <https://diveintosystems.org>

Course website: <https://brynmawr-cs223-f26.github.io/website>

Slack: <https://brynmawr-cs223-f26.slack.com>

Github: <https://github.com/BrynMawr-CS223-F26>

Heads Up

This class wasn't added until July, so the roster isn't set yet.

First lab: setting up ssh accounts, github accounts, github ssh keys.

First week: Access to your private homework repo set up.

First homework: Won't be due until the end of week 2.

Look Out for Invites!

Later this week: look out for an invitation to a **private github repo** under our github org.

It will be of this form: `BrynMawr-CS223-F26/your-github-username-code`

Also look out for an invite to our class **slack** channel.

Announcements, questions, etc.

Course Format

Lectures

- Combination of slides + live coding / debugging

Labs

- Code practice + ask me any questions
- Worksheets (completed in pairs)
- Timed quizzes (individual)

Assignments

- Weekly (due on Fridays)

Exams

- Midterm exam
- Final exam

Grade Breakdown

Final Exam: 35%

Midterm Exam: 35%

Homework: 10%

Code Challenges / Quizzes: 15%

Lab Practice / Class Participation: 5%

Late Policy

No extensions.

Assignments will only be accepted for 48 hours after the deadline.

Each day the assignment is late incurs a penalty of 20%.

Accommodations

Please provide at least 2 weeks prior notice for extensions on quizzes and exams.

Sick Policy

Prof Shiptoski is on **immunosuppressing medications**.

PLEASE wear a **mask** if you are feeling **sick** or experiencing symptoms.

Office hours will be **flexible**. If you are sick and need help, we can set up a **Zoom** call.

If you are sick and need to miss a quiz or lab, contact me to **reschedule it**.

Programming Assignments

Submissions MUST compile using make on our UNIX systems.

- Test on our servers before you submit.
- Do not change base code!

To receive full credit:

- Follow the class coding style.
- Appropriate comments and testing.
- No memory errors (test with valgrind).

Assignments will be completed by you alone. No group work.

How to Succeed

- Do the work.
- Do not procrastinate.
- Do not expect the C programming language to come naturally to you.
- Attend lectures and labs.
- **ASK QUESTIONS!**
 - Labs / Lectures / Office Hours / By appointment.
 - Slack (we will try to respond within 24 hours, M-F).

Seriously, ask me anything. I am here to help!

Some Advice

Systems programming is **hard**. Arguably some of the hardest code to write.

Chances are, you won't be great at it when you start. (I certainly wasn't.)

Don't give up.

Effort is heavily considered in the assessment of all assignments and labs.

Ask me about anything and everything that confuses you.

This Week's Goals

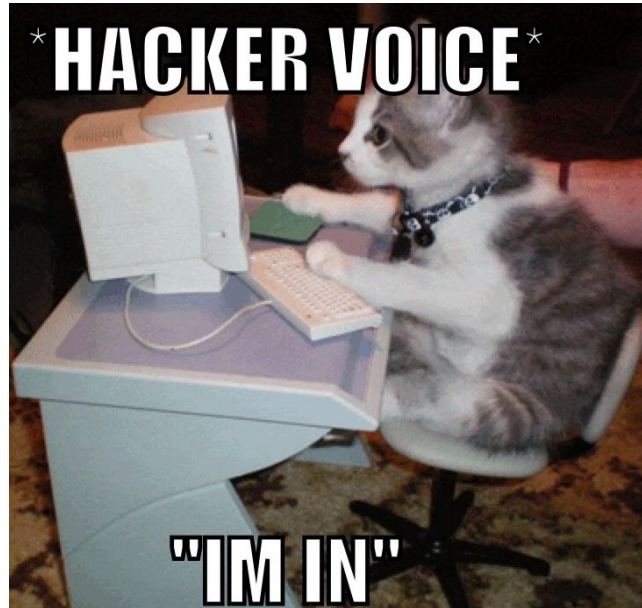
Set up your **UNIX** account.

Sign into **slack**.

Set up **SSH** and **Github**.

Pull and **push** to your code repository.

Let's Get Started!



What do I mean by “Systems”?

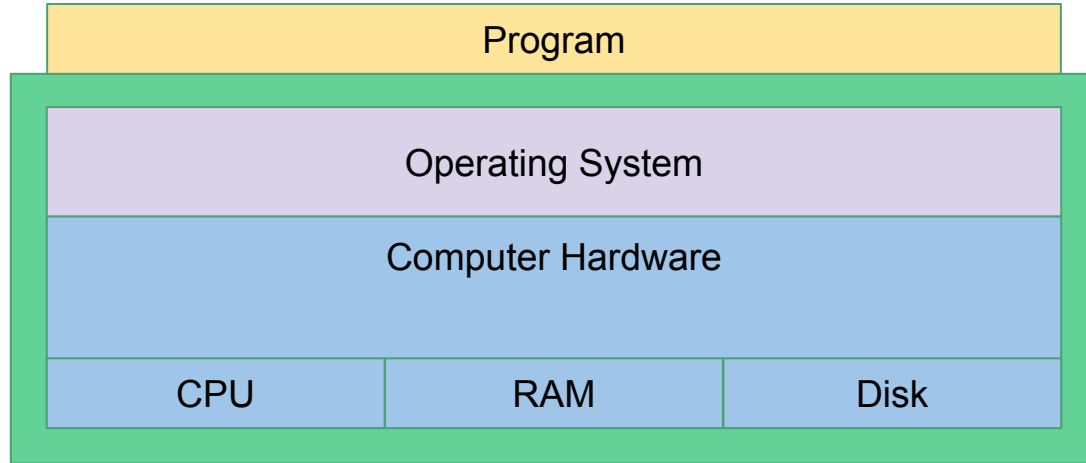
Hardware and Special Systems Software (OS).

Work together to run application programs.

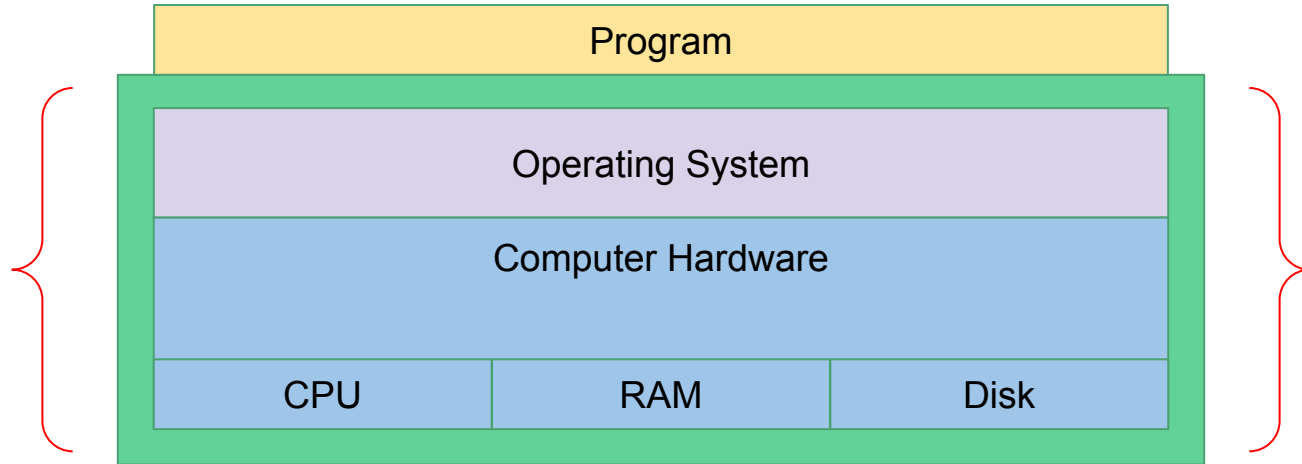
Hardware executes program instructions.

OS implements interface to user programs **and** manages hardware.

Computer System



Computer System



Systems Programming

- Programs that interact with hardware.
 - Drivers, hardware management, embedded systems.
- Tools intended for use by other programmers or software.
 - Compilers, operating systems, game engines, inter-process communication.

Typically **performance** or **memory** critical!

Course Topics

- C programming language fundamentals.
- Memory management.
- Debugging tools (gdb, valgrind, etc).
- Data representations “under-the-hood”.
- Working with UNIX, git, and terminal editors.
- Core operating systems concepts.
- Concurrent programming (processes and threads).
- Performance tradeoffs.
- Introduction to the Rust programming language.

Course Topics

- C programming language fundamentals.
- Memory management.
- Debugging tools (gdb, valgrind, etc).
- Data representations “under-the-hood”.
- Working with UNIX, git, and terminal editors.
- **Core operating systems concepts.**
- **Concurrent programming (processes and threads).**
- Performance tradeoffs.
- Introduction to the Rust programming language.

Binary Representations & Machine Code

- How to directly access and and modify memory.
- How data is stored in memory.
- What low-level machine instructions look like and how to read them.
 - x86_64 assembly

Let's Talk Performance Overhead

Question: why is my program running slowly?

Possibility #1: picked a bad algorithm (think big O).

Possibility #2: program is using systems resource inefficiently.

Memory utilization has a **huge impact** on program performance.

Example

Merge sort is $O(n \log n)$.

BUT: each merge pass requires moving elements from one list to another.

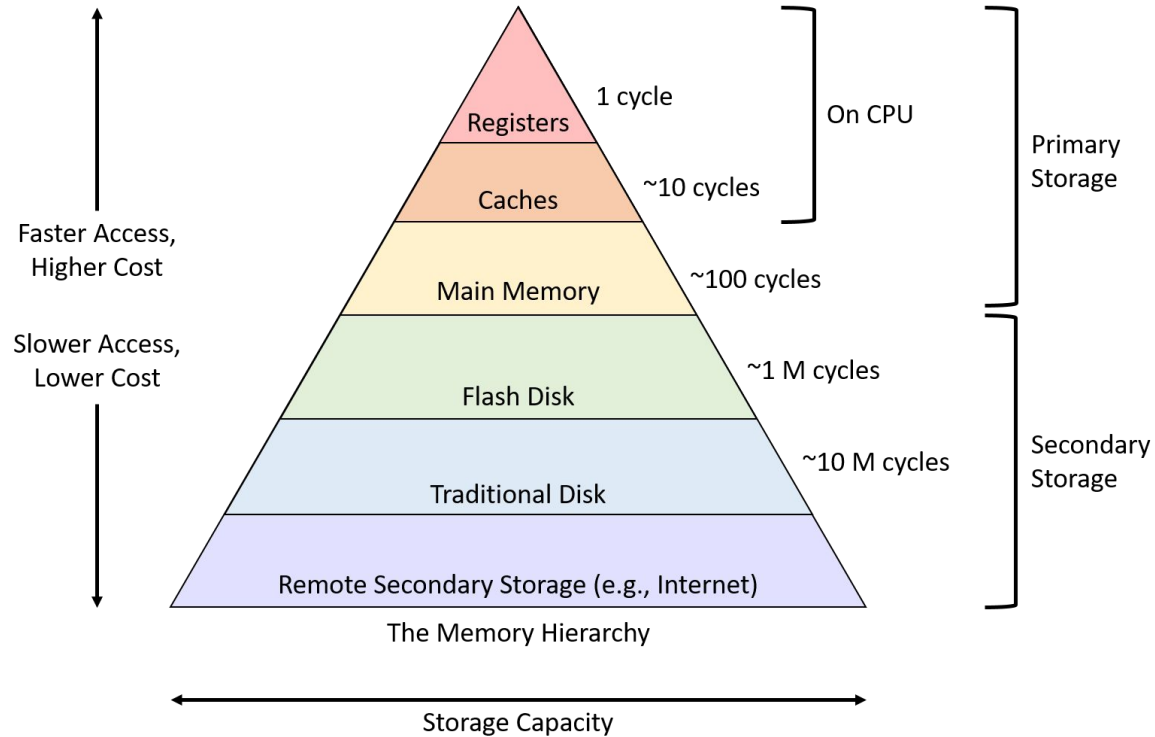
Requires $2n$ memory space.

Memory Hierarchy

We can't just focus on a program's runtime complexity.

We have to consider its memory complexity (memory access patterns).

Memory Hierarchy



Why Bother?

To understand the systems that form the **basis of all modern computing**.

To become **better** programmers:

- Write more **efficient** code.
- Understand the **limitations** of computing in terms of security, performance, energy, etc.

Skills to **design, build, and customize** your own software and devices.

Because it's fun!

Development Environment

The **platform** and **tools** you use to write software.

Systems programmers need to be able to:

- Work from the terminal using shell commands.
- Program in low level languages.
- Use debugging and profiling tools.

Development Environment

In this course:

- OS: Ubuntu Linux.
- Programming languages: C, x86_64 assembly, a little bit of Rust.
- Editor: vim, emacs, or nano (I use vim, and can therefore only be helpful with vim).
- Makefiles for compiling and linking.
- Git for version control.

C

Imperative (sequence of statements).

Procedural (structured using functions).

No classes.

Built in types such as strings and arrays.

Less abstracted than other languages.

- Easier to see relationship between code and its actual execution.
- Capable of more efficient code.
- Significantly more **dangerous** to write.

Java vs C

```
class Hello {  
    public static void main(String[] args) {  
        System.out.println(x: "Hello World!");  
    }  
}
```

```
#include <stdio.h>  
  
int main(int argc, char **argv) {  
    printf("Hello World!\n");  
    return 0;  
}
```

Java vs C

To compile:

```
gcc hello.c -o hello
```

```
javac Hello.java
```

To run:

```
./hello
```

```
java Hello
```

Building and Running a C Program

```
#include <stdio.h>

int main(int argc, char **argv) {
    printf("Hello World!\n");
    return 0;
}
```



compiler
(gcc, clang)



Binary executable:

```
01010110100110
11010010101001
00111011011001
11001010101001
10010010010011
00001100
```

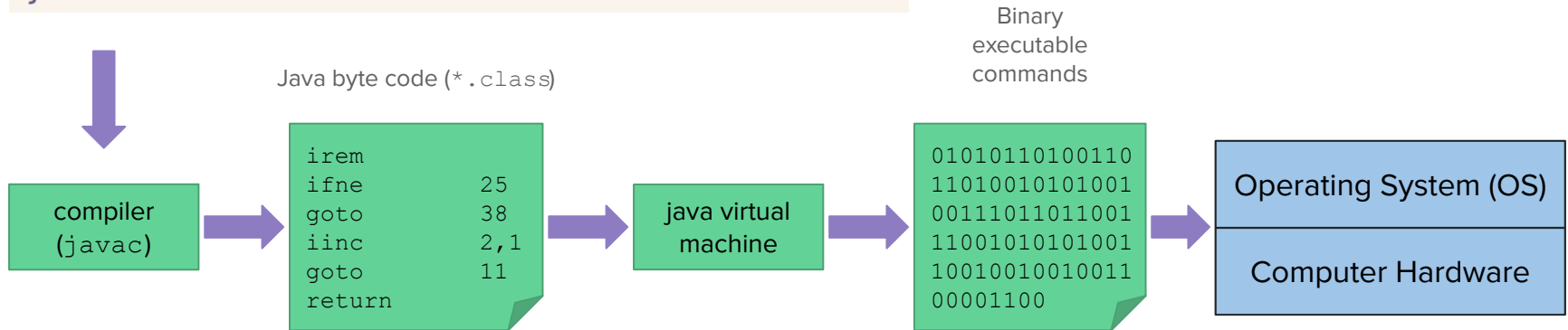


Operating System (OS)

Computer Hardware

Building and Running a **Java** Program

```
class Hello {  
    public static void main(String[] args) {  
        System.out.println(x: "Hello World!");  
    }  
}
```



All programs must be translated to binary in order to run on a computer.

Higher level languages (like Java) have more layers of abstraction (and therefore more steps from code → binary).

Binary code is specific to the hardware.

Languages can be cross-platform: the same program can run on different hardware (Linux, Mac, Windows, etc).

Compiles to different binary on different machines (OS + hardware).

This is nice and simple.

```
gcc hello.c -o hello
```

This is nice and simple.

```
gcc hello.c -o hello
```

What about something more complicated?

```
gcc -g -Wall -Werror -O2 hello.c -o hello
```

Simple Makefile Example

```
vim Makefile
```

```
make hello
```

```
hello: hello.c  
> gcc hello.c -o hello  
  
all: hello  
  
clean:  
> rm hello
```

Makefiles

```
hello: hello.c  
> gcc -g -Wall -Werror -O2 hello.c -o hello  
  
all: hello  
  
clean:  
> rm hello
```

UNIX Basics

We will mostly use **command line interfaces** (CLIs).

Terminal: text-based interface for the OS.

Command line: current line in the terminal, where a command is issued.

Command prompt: prefix text at the beginning of the command line.

Shell: program that executes commands from terminal (bash, zsh, powershell...).

Exercise: Connect to Server

On a laptop or desktop computer, open a terminal and `ssh` to comet.

```
ssh <username>@comet.cs.brynmawr.edu
```

Exercise: Create and Edit a File

Write and compile a program (hello.c) that prints “Hello World!”.

```
vim hello.c
```

```
gcc hello.c  
./a.out
```

```
gcc hello.c -o hello  
./hello
```

```
#include <stdio.h>  
  
int main(int argc, char **argv) {  
    printf("Hello World!\n");  
    return 0;  
}
```

Useful Commands

`ls` - list all directories.

`cd`, `mkdir`, `pwd` - change directory, make directory, print working directory.

`mv`, `cp`, `rm` - move, cp, remove.

`touch` - create empty file.

`cat`, `less`, `more` - show files.

`javac`, `gcc`, `make` - compiling programs.

Useful Commands

`grep, find` - searching files.

`man` - read documentation.

`ssh <username>@<server>.cs.brynmawr.edu` - log into CS server.

`git` - source control.

Working with Paths

Path: route through which the file can be accessed on the computer, unique address of a file.

Absolute paths: path from root of file system to the file.

Relative paths: path from **current working directory** to the file.

File extension: tells OS what type of data the file contains (ex: *.txt, *.jpg, *.c).

Special Directories

<code>..</code>	Parent directory
<code>.</code>	Current working directory
<code>/</code>	Root directory
<code>/home/username</code>	Your home directory
<code>~</code>	Your home directory

Path Examples

Relative path examples:

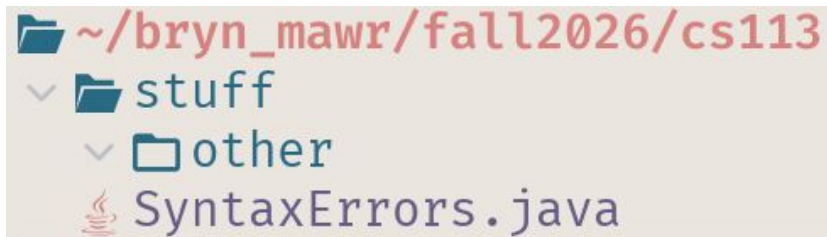
```
../../cs113
```

```
./hello.c
```

Absolute path example:

```
/Users/kellyshiptoski/bryn_mawr/fall2026/cs223/lecture2
```

Exercise



```
~/bryn_mawr/fall2026/cs113
├── stuff
│   └── other
│       └── SyntaxErrors.java
```

```
/Users/kellyshiptoski/bryn_mawr
```

What is the absolute path of SyntaxErrors.java?

What is the relative path of SyntaxErrors.java from:

- The root directory?
- The stuff directory?
- The other directory?

Exercise

Visualize the directory hierarchy after running the following commands:

```
> pwd
/home/kelly
> mkdir one
> cd one
> mkdir two
> touch talk.c
> cd ..
> touch listen.c
> cd
> touch sing.c
```

Terminal Editors

Necessary for this course (in labs, homework, etc).

Options:

- Emacs
- Nano
- Vim
 - **Neovim!**

Note: I use and know **Vim**, I am not an Emacs or Nano expert. Use at your own risk!

Basic Vim Commands

<code>vim <filename></code>	Open file
<code>i</code>	Enter insert mode
<code>esc</code>	Exit current mode
<code>esc + :q!</code>	Quit (without saving)
<code>esc + :w</code>	Save
<code>esc + :wq</code>	Save and quit

From Java to C

Agenda

Syntax

Stack diagrams

Built-in Types

Console I/O

Arrays

Strings

Structs

Java → C

Syntax is similar.

- `if/else` statements, loops (`while/for`), variables, functions
- `{ }` for statement blocks
- `;` for statements
- Operators and operator precedence
 - `+`, `-`, `*`, `/`, `+=`, `&&`, `||`, `%`, etc

Data types are similar.

- `int` (`1`, `-5`, `0`)
- `double`, `float` (`1.0`, `-4.7`)
- `char` (`'a'`, `'$'`, `'\n'`)

The Stack

Execution stack: keeps track of active functions.

We will use stack diagrams to visualize program flow.

For checking your work: <https://pythontutor.com/c.html>.

Stack Frames

A stack frame is created for each function.

- Contains parameters, local variables.
- Topmost frame is currently executing function.
- Frame is pushed to stack when function is called.
- Frame is popped off stack when it returns.

Java → C

What is the output of this program?

```
#include "stdio.h"

int func(int a, int b) {
    a += 5;

    if (a > 10) {
        return 0;
    } else {
        return a - b;
    }
}

int main() {
    int x, y;
    x = 4;
    y = 7;

    for (int i = 0; i < 3; i++) {
        y = func(a: x, b: y);
        printf("%d, %d\n", x, y);
    }
}
```

(Signed) Numeric Types in C

Type name	Usual size	Values stored	How to declare
char	1 byte	integers	char x;
short	2 bytes	signed integers	short x;
int	4 bytes	signed integers	int x;
long	4 or 8 bytes	signed integers	long x;
long long	8 bytes	signed integers	long long x;
float	4 bytes	signed real numbers	float x;
double	8 bytes	signed real numbers	double x;

Questions

What exactly is a **byte**?

Why might we want to use:

- An `int` instead of a `long`?
- A `float` instead of a `double`?
- A `long` instead of an `int`?
- A `double` instead of a `float`?

When would we use an **unsigned** variant over a **signed** one?

Print Functions

C: `printf()`

Java: `System.out.printf()`

```
printf("%d %s\t%f\n", 6, "hello", 3.4);
```

```
System.out.printf(format: "%d %s\t%f\n", ... args: 6, "hello", 3.4);
```

Both functions output: `6 hello 3.400000`

<code>%d, %i</code>	int
<code>%f</code>	float
<code>%g</code>	double
<code>%s</code>	string
<code>%lu</code>	long unsigned int
<code>%x</code>	unsigned hexadecimal
<code>\t</code>	tab character
<code>\n</code>	new line character

For future reference: <https://www.cplusplus.com/reference/cstdio/printf/>

Input

`scanf()` : for reading in values of different types.

Uses format string like `printf()`.

The arguments are read into an array.

The arguments are the memory locations of the stored values.

scanf

The arguments are read into an array.

The arguments are the memory locations of the stored values.

```
char s[100];  
int x;  
float y;  
  
scanf("%s %d %f", s, &x, &y);
```

s is the base address of the string array.

&x is the **address of the variable x** in memory.

&y is the **address of the variable y** in memory.

Example

```
int main() {  
    printf("Do you like jokes? (y/n): ");  
  
    char response = 0;  
    scanf("%c", &response);  
  
    printf("Your response: %c\n", response);  
  
    if (response == 'y') {  
        printf("Me too!\n");  
    } else if (response == 'n') {  
        printf("Me neither!\n");  
    } else {  
        printf("Are directions difficult for you?\n");  
    }  
  
    return 0;  
}
```

Booleans

There are **no booleans** in C.

0 is always **false**.

Every other value is **true**.

Helpful to treat **0 as false** and **1 as true**.

Example

```
int isEven(int val) {  
    int mod = val % 2;  
    printf("%d is val of mod\n", mod);  
    if (mod == 0) {  
        return 1;  
    }  
  
    return 0;  
}
```

```
int main() {  
    int a = 0;  
    int b = 7;  
  
    if (isEven(val: a)) {  
        printf("a is even\n");  
    } else {  
        printf("a is odd\n");  
    }  
  
    if (b) {  
        printf("Any non-zero value is true!\n");  
    }  
}
```

Arrays: Java vs C

```
class Array {
    public static void main(String[] args) {
        int i, size = 0;
        int[] arr = new int[10];

        for (i = 0; i < 10; i++) {
            arr[i] = i;
            size++;
        }

        arr[3] = 100;

        System.out.printf(format: "array of %d items:\n", size);

        for (i = 0; i < 10; i++) {
            System.out.printf(format: "%d\n", arr[i]);
        }
    }
}
```

```
int main() {
    int i, size = 0;
    int arr[10];

    for (i = 0; i < 10; i++) {
        arr[i] = i;
        size++;
    }

    arr[3] = 100;
    printf("array of %d items:\n", size);

    for (i = 0; i < 10; i++) {
        printf("%d\n", arr[i]);
    }

    return 0;
}
```

Arrays in C

In C, the size of an array is set at compile time and **cannot change during runtime**.

Other languages, like Java, provide both **statically** and **dynamically** allocated arrays.

- Statically allocated: `Java.util.Arrays`
- Dynamically allocated: `Java.util.ArrayList`

Arrays in C

There is **no bounds checking** in C.

It is very easy to write/read elements out of bounds of an array.

You may (or may not) notice the problem, depending on what memory you access...

Out of Bounds Access: C

```
int main() {  
    int bad[20] = {[0]=1, [1]=2, [2]=3};  
  
    printf("bad[2] = %d\n", bad[2]);  
    printf("bad[20] = %d\n", bad[20]);  
    return 0;  
}
```

```
> ./out-of-bounds
```

```
bad[2] = 3
```

```
bad[20] = -1195769735
```



Out of Bounds Access: Java

```
class BadArray {  
    public static void main(String[] args) {  
        int[] bad = { 1, 2, 3, 4 };  
  
        System.out.printf(format: "bad[2] = %d\n", bad[2]);  
        System.out.printf(format: "bad[4] = %d\n", bad[4]);  
    }  
}
```

```
bad[2] = 3
```

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 4 out of bounds for length 4
```

Exercise: Fibonacci

Write a C program that computes the first 10 Fibonacci numbers and stores them in an array.

Strings in C

```
// Array of chars  
char greeting[5] = "ciao";  
  
// Pointer to read-only array of chars.  
const char *message = "This is the breakdown";
```

Strings in C

In C, a **string** is an **array of chars**.

Not every **array of chars** is a **string**, but every **string** is an **array of chars**.

Strings in C

Strings in C **must** be large enough to hold the **number of characters + 1**.

Last character is a **null character** that denotes the end of the string (`'\0'`).

C strings are **null-terminated**.

```
#include <stdio.h>
#include <string.h>

int main() {
    char greeting[5] = "ciao";
    const char *message = "This is the breakdown";

    char buffer[10];
    buffer[0] = 'h';
    buffer[1] = 'i';
    buffer[2] = '\0';

    printf("%s\n", greeting);
    printf("%s\n", message);
    printf("%s\n", buffer);

    int len = strlen(message);
    printf("Len of message is: %d\n", len);

    strcpy(buffer, "test");
    printf("Buffer is: %s\n", buffer);

    return 0;
}
```

```
ciao
This is the breakdown
hi
Len of message is: 21
Buffer is: test
```

String Library in C

```
#include <string.h>
```

strlen	Length of string
strcpy	Copy
strcat	Concatenate
strcmp	Compare

Let's do some examples.

strlen

```
const char *message = "Hi there";  
  
int len = strlen(s: message);  
printf("Len of message is: %d\n", len);
```

```
Len of message is: 8
```

`strlen` returns the number of characters in the string, not including the null terminator.

strcpy

```
char buffer[10];  
buffer[0] = 'h';  
buffer[1] = 'i';  
buffer[2] = '\\0';  
  
strcpy(buffer, "there");  
printf("Buffer is: %s\\n", buffer);
```

```
Buffer is: there
```

strcpy copies data from one string into the memory of another string.

strcat

```
char buffer[10];  
buffer[0] = 'h';  
buffer[1] = 'i';  
buffer[2] = '\\0';  
  
strcat(buffer, "there");  
printf("Buffer is: %s\\n", buffer);
```

```
Buffer is: hithere
```

`strcat` joins two strings by concatenating a source string to the end of destination string.

Structs in C

No classes in C.

To bundle data together, we use a **struct**.

In Java, classes hold data **and** define methods.

In C, **structs** hold data and **functions** define methods.

Example

```
struct studentT {
    char name[64];
    int age;
    float gpa;
    int grad_yr;
};

int main() {
    struct studentT student1, student2;

    strcpy(student1.name, "Duncan Idaho");
    student1.age = 18 + 2;
    student1.gpa = 3.5;
    student1.grad_yr = 2020;

    // %s - string
    // %d - int
    // %g - float
    printf("name: %s, age: %d, gpa: %g, year: %d\n", student1.name, student1.age,
        student1.gpa, student1.grad_yr);

    return 0;
}
```

Typedef for C Structs

```
typedef struct studentT {
    char name[64];
    int age;
    float gpa;
    int grad_yr;
} studentT;

int main() {
    studentT student1, student2;

    strcpy(student1.name, "Duncan Idaho");
    student1.age = 18 + 2;
    student1.gpa = 3.5;
    student1.grad_yr = 2020;

    // %s - string
    // %d - int
    // %g - float
    printf("name: %s, age: %d, gpa: %g, year: %d\n", student1.name, student1.age,
        student1.gpa, student1.grad_yr);

    return 0;
}
```

Arrays and Structs in Memory

```
struct dataT {  
    int v[3];  
    char c;  
    float y;  
};  
  
int main() {  
    int arr[3];  
    arr[0] = -1;  
    arr[1] = 5;  
    arr[2] = 3;  
  
    struct dataT data;  
    data.v[0] = 2;  
    data.v[1] = -3;  
    data.v[2] = 0;  
    data.c = 'g';  
    data.y = 3.14;  
  
    for (int i = 0; i < 3; i++) {  
        printf("%d\n", data.v[i]);  
    }  
  
    return 0;  
}
```

Stack Diagram Examples

On board